

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA AN TOÀN THÔNG TIN

BÁO CÁO CHUYÊN ĐỀ

An toàn Thông tin

Đề tài

**Tái lập và mở rộng DeSFAM: phát hiện bất thường
lời gọi hệ thống cho container đám mây
bằng eBPF và học máy**

Một nghiên cứu tái lập trên bộ dữ liệu DongTing
kèm bộ phát hiện trực tiếp qua Tetragon trên Kubernetes

Giảng viên hướng dẫn : TS. Nguyễn Văn Dũng

Học viên thực hiện : Nguyễn Ngọc Hoài — CHAT4P006

Lớp : CHAT4P

Hà Nội, tháng 6 năm 2026

Tóm tắt

Container chia sẻ nhân hệ điều hành của máy chủ nên lời gọi hệ thống (syscall) trở thành bề mặt tấn công trọng yếu cho leo thang đặc quyền và thoát container. DeSFAM [1] là một framework bảo mật thời gian thực kết hợp lập hồ sơ syscall lai, phát hiện bất thường bằng học máy (SyscallAD: VAE + Rừng cô lập) và thực thi trong nhân bằng eBPF/LSM. Báo cáo chuyên đề này *tái lập* lỗi phát hiện bất thường của DeSFAM trên bộ dữ liệu công khai DongTing [2] và *mở rộng* nó thành một bộ phát hiện chạy trực tiếp: cảm biến syscall dùng Tetragon [3] (eBPF) trên cụm Kubernetes, đẩy cảnh báo qua Prometheus/Grafana. Toàn bộ quy trình huấn luyện và suy luận được đóng gói bằng Docker để bảo đảm khả năng tái lập. Trên DongTing, mô hình VAE đạt AUC kiểm định 0,944; tập hợp VAE + iForest đạt AUC kiểm định $\approx 0,95$ và AUC trên tập kiểm tra 0,835 (F1 0,417 tại ngưỡng bảo thủ chọn trên tập kiểm định). Khi triển khai trực tiếp, điểm bất thường tách biệt rõ giữa khối lượng công việc lành tính ($\leq 0,19$) và các kịch bản tấn công trong không gian tên thử nghiệm (0,278–0,930), cho phép một ngưỡng vận hành $T_0 = 0,25$ phát hiện tấn công thời gian thực. Báo cáo trình bày trung thực những điểm tái lập được, những chỗ lệch so với số liệu công bố của bài báo gốc, cùng các bài học về căn chỉnh không gian đặc trưng giữa huấn luyện và phục vụ.

Từ khóa: bảo mật container, lời gọi hệ thống, eBPF, Tetragon, phát hiện bất thường, Bộ mã hóa biến phân tự động (VAE), Rừng cô lập, DongTing, nghiên cứu tái lập.

Mục lục

1	Giới thiệu	1
1.1	Bối cảnh và động cơ	1
1.2	Mục tiêu của chuyên đề	1
1.3	Câu hỏi nghiên cứu	1
1.4	Đóng góp	2
2	Nền tảng và công trình liên quan	2
2.1	Bảo mật lời gọi hệ thống cho container	2
2.2	Sinh danh sách syscall và lọc động	2
2.3	Phát hiện xâm nhập dựa trên chuỗi syscall	2
2.4	Bộ dữ liệu DongTing	3
2.5	eBPF và Tetragon	3
2.6	Vị trí của chuyên đề	3
3	Phương pháp tái lập	3
3.1	Loại hình nghiên cứu	3
3.2	Kiến trúc tổng thể	3
3.3	Dữ liệu và tiền xử lý	3
3.4	Trích đặc trưng theo cửa sổ trượt	4
3.5	Mô hình SyscallAD	5
3.6	Chọn ngưỡng (không rò rỉ)	5
3.7	Mở rộng: bộ phát hiện trực tiếp	5
4	Thiết lập thực nghiệm	6
4.1	Nguyên tắc: mọi công cụ chạy bằng Docker	6
4.2	Huấn luyện ngoại tuyến	6
4.3	Cấu hình tái lập (ghi trong <code>results.json</code>)	6
4.4	Triển khai trực tiếp trên Kubernetes	7
4.5	Khối lượng công việc và kịch bản tấn công	7
4.6	Quan sát	7
5	Kết quả	7
5.1	CH1 — Tái lập ngoại tuyến trên DongTing	7
5.1.1	Hiệu năng từng mô hình và tập hợp	7
5.1.2	Đối chiếu với số liệu công bố	7
5.2	CH2 — Triển khai trực tiếp trên Tetragon/Kubernetes	8

5.3	CH3 — Điều kiện tái lập thành công	9
6	Thảo luận	9
6.1	Điều gì tái lập được, điều gì không	9
6.2	Ý nghĩa của khoảng cách ngưỡng	10
6.3	Lựa chọn thiết kế khác bài báo	10
6.4	Đe dọa tính hợp lệ	10
6.5	Hướng cải thiện	10
7	Kết luận	11
A	Phụ lục Tái lập Kết quả	13
A.1	Môi trường	13
A.2	Huấn luyện ngoại tuyến (tái lập Bảng 2)	14
A.3	Triển khai trực tiếp (tái lập Bảng 4)	14
A.4	Kiểm tra tính nhất quán đặc trưng	14
A.5	Biên dịch báo cáo này (Docker)	14

Danh mục hình vẽ

1	Kiến trúc DeSFAM được tái lập: cảm biến syscall (eBPF/Tetragon) → trích đặc trưng theo cửa sổ trượt → SyscallAD (VAE + iForest) → chấm điểm rủi ro và phản ứng. Mũi tên ngoại tuyến: huấn luyện trên DongTing; mũi tên trực tuyến: phục vụ trên Kubernetes.	4
2	Quy trình kỹ thuật đặc trưng: từ chuỗi syscall thô → lọc theo bảng chữ cái cảm biến → cửa sổ trượt (15, 3) → véc-tơ 43 chiều.	5
3	Phân tách điểm bất thường khi phục vụ trực tiếp qua Tetragon trên Kubernetes. Khối lượng lạnh tính (MongoDB) nằm dưới ngưỡng vận hành $T_0 = 0,25$, các kịch bản tấn công nằm trên—khoảng trống đủ rộng để một ngưỡng duy nhất tách sạch hai lớp.	9

Danh mục bảng

1	Cấu hình thí nghiệm tái lập (trích <code>results.json</code>).	6
2	Hiệu năng SyscallAD trên tập kiểm tra DongTing (cửa sổ 15/3, 43 chiều). Ngưỡng chọn trên tập kiểm định.	8
3	Tái lập so với số liệu công bố của DeSFAM (lỗi phát hiện bất thường). .	8
4	Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon/K8s). . . .	9

1 Giới thiệu

1.1 Bối cảnh và động cơ

Công nghệ container đã trở thành nền tảng của điện toán đám mây hiện đại nhờ tính nhẹ và khả năng mở rộng. Khác với máy ảo, container chia sẻ nhân hệ điều hành của máy chủ; điều này giảm chi phí vận hành nhưng cũng tạo ra rủi ro bảo mật nghiêm trọng tại giao diện lời gọi hệ thống (syscall)—ranh giới giữa không gian người dùng và nhân [4, 5]. Kẻ tấn công lạm dụng syscall để leo thang đặc quyền, thoát container hoặc tiêm chuỗi lời gọi độc hại [6]. Các cơ chế tĩnh như hồ sơ seccomp khó duy trì, dễ chặn nhầm thao tác hợp lệ hoặc bỏ sót mối đe dọa nhiều bước.

Bài báo DeSFAM [1] đề xuất một hướng tiếp cận thích nghi: kết hợp lập hồ sơ syscall lai (tĩnh + động), phát hiện bất thường bằng học máy có độ trễ thấp (mô-đun *SyscallAD* dùng Bộ mã hóa biến phân tự động VAE và Rừng cô lập iForest), tính điểm rủi ro theo ngữ cảnh (ánh xạ MITRE ATT&CK và tương quan CVE), và thực thi trong nhân qua eBPF/LSM. Đây là một thiết kế hấp dẫn nhưng bài báo gốc đánh giá trên hạ tầng riêng và không công bố mã nguồn đầy đủ, nên *khả năng tái lập* của nó cần được kiểm chứng độc lập.

1.2 Mục tiêu của chuyên đề

Chuyên đề này không đề xuất một phương pháp mới mà thực hiện một *nguyên cứu tái lập* (reproduction study) có kiểm soát đối với phần lõi của DeSFAM, đồng thời *hiện thực hóa* thành phần thực thi/quan sát trực tiếp mà bài báo mô tả nhưng không phát hành. Cụ thể, chúng tôi:

- (i) Tái lập mô-đun phát hiện bất thường SyscallAD (VAE + iForest, hợp nhất tập hợp) trên bộ dữ liệu công khai DongTing [2];
- (ii) Xây dựng một bộ phát hiện *trực tiếp*: thu thập syscall bằng Tetragon [3] (eBPF) trên Kubernetes, trích đặc trưng theo cửa sổ trượt giống hệ lúc huấn luyện, chấm điểm và phát cảnh báo qua Prometheus/Grafana;
- (iii) Đối chiếu trung thực kết quả tái lập với số liệu công bố và phân tích nguyên nhân các điểm lệch.

1.3 Câu hỏi nghiên cứu

- **CH1 (Khả năng tái lập ngoại tuyến):** Trên bộ dữ liệu DongTing, bộ phát hiện VAE + iForest có đạt được hiệu năng phân loại tiệm cận mức công bố của DeSFAM hay không?
- **CH2 (Khả năng triển khai trực tiếp):** Mô hình huấn luyện ngoại tuyến có giữ được khả năng phân tách lành tính/tấn công khi phục vụ trực tiếp trên luồng syscall do Tetragon sinh ra trong cụm Kubernetes không?
- **CH3 (Điều kiện thành công):** Yếu tố kỹ thuật nào quyết định việc tái lập thành công—đặc biệt là sự *căn chỉnh không gian đặc trưng* giữa giai đoạn huấn luyện và giai đoạn phục vụ?

1.4 Đóng góp

- Một quy trình tái lập SyscallAD *hoàn toàn bằng Docker*, có thể chạy lại từ đầu, với kết quả số được lưu vết (`results.json`).
- Một bộ phát hiện trực tiếp dựa trên Tetragon/eBPF trên Kubernetes, chứng minh thiết kế DeSFAM khả thi ngoài môi trường mô phỏng.
- Một báo cáo *trung thực* về khoảng cách giữa kết quả tái lập và số liệu công bố, cùng bài học về căn chỉnh “bảng chữ cái cảm biến” (sensor alphabet) giữa huấn luyện và phục vụ.

Phần còn lại được tổ chức như sau: Mục 2 trình bày nền tảng và công trình liên quan; Mục 3 mô tả phương pháp tái lập; Mục 4 trình bày thiết lập thực nghiệm bằng Docker/Kubernetes; Mục 5 báo cáo kết quả; Mục 6 thảo luận và nêu các đe dọa tính hợp lệ; Mục 7 kết luận.

2 Nền tảng và công trình liên quan

2.1 Bảo mật lời gọi hệ thống cho container

Trong mô hình chia sẻ nhân, nhân Linux phơi bày hàng trăm syscall, phần lớn không cần thiết cho một khối lượng công việc cụ thể, làm phình to bề mặt tấn công [7]. Các cơ chế truyền thống (seccomp, AppArmor, SELinux) lọc theo chính sách tĩnh nên: (1) không thích nghi với hành vi thời gian chạy; (2) thiếu nhận thức về *chuỗi* syscall nên mù với tấn công nhiều bước; và (3) tốn chi phí cấu hình thủ công ở quy mô lớn.

2.2 Sinh danh sách syscall và lọc động

Confine [8] dùng phân tích tĩnh để rút gọn hồ sơ seccomp ($SRR \approx 47,6\%$) nhưng không thích nghi thời gian chạy. Prof-gen [9] kết hợp theo dõi lúc khởi động với đồ thị lời gọi tĩnh, vẫn bỏ sót thay đổi hành vi về sau. Optimus [10] và KubeRosy [11] dùng eBPF để cập nhật chính sách thời gian chạy, song Optimus cần phê duyệt thủ công và KubeRosy gắn chặt với Kubernetes. NIMOS [12] đánh giá giá trị của lọc dựa trên chuỗi. DeSFAM định vị mình ở chỗ hợp nhất cả ba: lập hồ sơ lai, phát hiện bất thường học máy, và thực thi trong nhân.

2.3 Phát hiện xâm nhập dựa trên chuỗi syscall

Hướng phát hiện bất thường dựa trên chuỗi syscall có lịch sử lâu dài, khởi nguồn từ “a sense of self” [13, 14] và các mô hình thay thế [15], cùng cảnh báo kinh điển về tấn công bất chước [16]. Các hướng hiện đại gồm phương pháp ngữ nghĩa [17], túi lời gọi (BoSC) cho container [18], và học sâu một lớp/biến phân như DAGMM [19], Deep SVDD [20], HIDS học sâu [21]. Riêng cho container có các đánh giá phát hiện bất thường [22] và ngữ cảnh hóa syscall [23], cũng như tập hợp Random/Isolation Forest [24]. SyscallAD của DeSFAM nằm trong dòng này: hợp nhất VAE [25] (lỗi tái cấu trúc) với iForest [26] (ngoại lệ thống kê), bổ sung khớp mẫu lãnh tính bằng PrefixSpan [27].

2.4 Bộ dữ liệu DongTing

DongTing [2] là bộ dữ liệu quy mô lớn gồm các chuỗi syscall của nhân Linux được gán nhãn bình thường/độc hại (theo các bản vá lỗi syzkaller), thích hợp cho huấn luyện và xác nhận bộ phát hiện bất thường. Chuyên đề này dùng DongTing làm nguồn dữ liệu ngoại tuyến duy nhất, đúng như DeSFAM.

2.5 eBPF và Tetragon

eBPF cho phép nạp logic giám sát/thực thi an toàn vào nhân tại các điểm vào syscall với chi phí thấp [28]. DeSFAM dùng eBPF/LSM để chặn nội tuyến. Trong chuyên đề này, thay vì tự viết chương trình eBPF, chúng tôi dùng Tetragon [3] (thuộc hệ sinh thái Cilium [29])—một lớp quan sát và thực thi bảo mật dựa trên eBPF cho Kubernetes—làm *cảm biến syscall* phát sự kiện qua gRPC. Đây là lựa chọn thực dụng để hiện thực hóa lớp thu thập của DeSFAM mà không cần sửa nhân.

2.6 Vị trí của chuyên đề

Khác với các công trình trên (đề xuất phương pháp mới), chuyên đề này là một *nguyên cứu tái lập*: kiểm chứng độc lập tính khả thi và hiệu năng của lõi DeSFAM, đồng thời lấp khoảng trống “từ mô phỏng đến triển khai” bằng một bộ phát hiện trực tiếp trên Tetragon/Kubernetes.

3 Phương pháp tái lập

3.1 Loại hình nghiên cứu

Đây là một *nguyên cứu tái lập có kiểm soát*: biến cần tái lập là hiệu năng phát hiện bất thường của SyscallAD, các yếu tố khác (bộ dữ liệu, định nghĩa đặc trưng, giao thức đánh giá) được cố định bám sát mô tả của bài báo gốc [1]. Phần mở rộng (bộ phát hiện trực tiếp) là một *nguyên cứu hiện thực hóa* nhằm kiểm chứng tính khả thi triển khai.

3.2 Kiến trúc tổng thể

Hình 1 mô tả kiến trúc ba giai đoạn của DeSFAM mà chúng tôi bám theo: (1) lập hồ sơ syscall lai; (2) phát hiện bất thường SyscallAD; (3) thực thi/phản ứng. Chuyên đề tập trung tái lập trọn vẹn Giai đoạn 2 và hiện thực hóa lớp thu thập/phản ứng của Giai đoạn 1 và 3 bằng Tetragon (cảnh báo, ghi nhật ký).

3.3 Dữ liệu và tiền xử lý

Nguồn dữ liệu ngoại tuyến là bản phát hành DongTing [2] (18,9 nghìn chuỗi được gán nhãn, các tệp `.log` kèm bảng phân chia `Baseline.xlsx`). Chuỗi bình thường lưu dưới dạng *ID syscall* phân tách bằng dấu gạch đứng; chuỗi tấn công lưu dưới dạng *tên syscall*. Chúng tôi dùng bảng `syscall_64.tbl` (x86-64, bỏ các dòng ABI x32) để ánh



Hình 1: Kiến trúc DeSFAM được tái lập: cảm biến syscall (eBPF/Tetragon) → trích đặc trưng theo cửa sổ trượt → SyscallAD (VAE + iForest) → chấm điểm rủi ro và phản ứng. Mũi tên ngoại tuyến: huấn luyện trên DongTing; mũi tên trục tuyến: phục vụ trên Kubernetes.

xạ tên ↔ ID nhất quán. Phân chia theo bộ dữ liệu: DTDS-train / DTDS-validation / DTDS-test.

Bảng chữ cái cảm biến (sensor alphabet). Đây là quyết định thiết kế *then chốt* để tái lập thành công khâu triển khai trực tiếp. Chính sách truy vết trực tiếp của Tetragon chỉ phát ra **23 syscall liên quan an ninh** (execve, clone, setuid/setgid/capset, openat, unlinkat, renameat2, mount, unshare, setns, pivot_root, socket, connect, bind, mprotect, mmap, prctl, memfd_create, splice, ptrace, init/finit_module, bpf—*không* gồm read/write/close/futex). Do đó khâu huấn luyện cũng lọc DongTing về đúng tập này *trước* khi cắt cửa sổ, để huấn luyện và phục vụ chia sẻ một không gian đặc trưng. Nếu huấn luyện trên toàn bộ vũ trụ syscall, mọi cửa sổ trực tiếp sẽ suy biến (lành tính $\approx$ tấn công).

3.4 Trích đặc trưng theo cửa sổ trượt

Mỗi cửa sổ trượt (độ dài 15, bước nhảy 3—giá trị mặc định) được biểu diễn thành véc-tơ **43 chiều** (Hình 2):

$$[\text{freq}_8 \mid \text{disc}_{15} \mid \text{stats}_8 \mid \text{cat}_{10} \mid \text{prefixspan}_2].$$

- **freq (8):** tần suất trong cửa sổ của các syscall phổ biến.
- **disc (15):** *hiện diện nhị phân* của các syscall đặc quyền hiếm (ptrace/splice/unshare/setns/mount, bên với pha loãng: một ptrace đơn lẻ trong cửa sổ vẫn bật bit).
- **stats (8):** entropy, số phần tử duy nhất, độ dài, tần suất lớn nhất...
- **cat (10):** tần suất chuẩn hóa theo nhóm chức năng (tiến trình, tệp, bộ nhớ, mạng, tín hiệu, thời gian, IPC, an ninh, sự kiện I/O, khác).
- **prefixspan (2):** [cờ-khớp, độ-dài-khớp/L] từ việc đối sánh cửa sổ với một *Cơ sở dữ liệu mẫu lành tính* khai phá bằng PrefixSpan [27] trên cửa sổ huấn luyện bình thường.

- Đặc trưng thời gian (Δt) bị *tắt* vì DongTing (`strace -v -f`) không có dấu thời gian theo từng syscall (`has_temporal=false`).

Chuẩn hóa bằng RobustScaler với phân vị (1,99), *chỉ* khớp trên cửa sổ huấn luyện bình thường để tránh rò rỉ dữ liệu tấn công.



Hình 2: Quy trình kỹ thuật đặc trưng: từ chuỗi syscall thô → lọc theo bảng chữ cái cảm biến → cửa sổ trượt (15, 3) → véc-tơ 43 chiều.

3.5 Mô hình SyscallAD

- **VAE** [25]: `hidden_dim=32, latent_dim=8`, huấn luyện *chỉ* trên cửa sổ bình thường; điểm bất thường là lỗi tái cấu trúc. Huấn luyện đa hạt giống (seed 0,1,2), giữ hạt giống có AUC kiểm định tốt nhất để xử lý hiện tượng *sụp đổ hậu nghiệm* (posterior collapse).
- **iForest** [26]: 300 cây, `contamination=0,02`, cũng huấn luyện trên dữ liệu bình thường.
- **Hợp nhất tập hợp**: $s = 0,7 \cdot \text{RobustScale}(s_{\text{VAE}}) + 0,3 \cdot \text{RobustScale}(s_{\text{iForest}})$.

3.6 Chọn ngưỡng (không rò rỉ)

Ngưỡng phát hiện được chọn *chỉ* trên tập kiểm định và lưu vào `results.json`; tập kiểm tra không tham gia chọn ngưỡng. Khi phục vụ trực tiếp, ngưỡng vận hành được hiệu chỉnh lại theo phân phối điểm quan sát trên cụm (xem Mục 5).

3.7 Mở rộng: bộ phát hiện trực tiếp

Thành phần suy luận đọc luồng sự kiện syscall từ Tetragon [3] qua gRPC, gom theo tiến trình, cắt cửa sổ (15, 3) và trích *đúng* véc-tơ 43 chiều như lúc huấn luyện (được kiểm tra ngang hàng bằng `tests/test_feature_parity.py`), rồi chấm điểm bằng các mô hình đã lưu. Cảnh báo và chỉ số được phơi bày qua Prometheus và trực quan hóa trên Grafana; nhật ký sự kiện Tetragon được thu thập qua Loki/Promtail. Toàn bộ chạy bằng Docker/Kubernetes, không cài công cụ trực tiếp lên máy chủ.

4 Thiết lập thực nghiệm

4.1 Nguyên tắc: mọi công cụ chạy bằng Docker

Để bảo đảm khả năng tái lập và không làm “bẩn” máy chủ, toàn bộ chuỗi công cụ (huấn luyện, suy luận, biên dịch báo cáo) đều chạy trong container. Quy trình huấn luyện được đóng gói bằng `docker-compose`; bộ phát hiện trực tiếp, Prometheus và Grafana cũng khởi chạy bằng `docker-compose`; cụm Kubernetes thử nghiệm cài Tetragon qua script.

4.2 Huấn luyện ngoại tuyến

Một nguồn chân lý duy nhất là `train.py` (mọi hàm học máy, không vẽ đồ thị); sổ tay `train_dongting.ipynb` nhập lại từ `train.py` và bổ sung đồ thị + bình luận học thuật. Hai chế độ luôn đồng bộ: muốn đổi thuật toán chỉ sửa `train.py`.

```
cd DeSFAM/training
# ạChy nhanh, không ươtng tác:
docker compose -f docker-compose.pipeline.yml run --rm train
# ặHoc ươtng tác (Jupyter Lab :8888):
docker compose -f docker-compose.pipeline.yml up jupyter
```

Đầu ra (lưu vào `../model/`) gồm: bộ chuẩn hóa đặc trưng, mô hình iForest, trọng số bộ mã hóa/giải mã VAE, các bộ chuẩn hóa điểm thành phần, tham số tập hợp ($\alpha = 0,7$), từ vựng đặc trưng (`fe_report.json`), và `results.json` (chỉ số + ngưỡng + siêu tham số).

4.3 Cấu hình tái lập (ghi trong `results.json`)

Bảng 1 liệt kê cấu hình thí nghiệm được lưu vết tự động, giúp người đánh giá đối chiếu chính xác.

Bảng 1: Cấu hình thí nghiệm tái lập (trích `results.json`).

Tham số	Giá trị
Thí nghiệm	<code>dongting_15_3</code>
Số chuỗi	18 874
Cửa sổ trượt (độ dài, bước)	(15, 3)
Số cửa sổ huấn luyện / kiểm định / kiểm tra	90 456 / 896 581 / 687 926
Số chiều đặc trưng	43
VAE latent_dim / hidden_dim	8 / 32
iForest contamination	0,02
Đặc trưng thời gian	không (<code>has_temporal=false</code>)
Số mẫu lành tính (PrefixSpan)	150

4.4 Triển khai trực tiếp trên Kubernetes

Cảm biến syscall là Tetragon [3], cài qua script và áp một chính sách truy vết (syscall-ad-tracing) chỉ phát ra 23 syscall an ninh (Mục 3). Bộ phát hiện đọc sự kiện qua gRPC NodePort.

```
cd DeSFAM/Kubernetes
bash tetragon/install.sh                # cài Tetragon + NodePort
kubectl apply -f tetragon/tracing-policy.yaml # chính sách 23 syscall
bash deploy.sh                          # workload lạnh tính + tan cong
```

```
cd DeSFAM/inference
# Detector + Prometheus + Grafana, dung nguong van hanh da hieu chinh:
docker compose --env-file .env.uat up -d --build
```

4.5 Khối lượng công việc và kịch bản tấn công

Khối lượng lạnh tính tiêu biểu là MongoDB (truy vấn CRUD bình thường). Kịch bản tấn công trong không gian tên demo mô phỏng các chuỗi đặc quyền (thực thi shell bất thường, ptrace, nạp mô-đun, thao tác mount/unshare) phản ánh các họ CVE thoát container và leo thang đặc quyền như CVE-2021-4034 [30], CVE-2022-0847 [31], CVE-2019-5736 [32]. Mục tiêu là quan sát *độ phân tách điểm* giữa lạnh tính và tấn công khi phục vụ trực tiếp, chứ không tuyên bố chặn 14/14 CVE như bài báo gốc (xem Mục 6).

4.6 Quan sát

Grafana (:3000) hiển thị điểm bất thường theo thời gian và nhật ký sự kiện Tetragon (qua Loki); Prometheus (:9090) lưu chuỗi thời gian của điểm số và cảnh báo. Đây là kênh bằng chứng cho CH2.

5 Kết quả

5.1 CH1 — Tái lập ngoại tuyến trên DongTing

5.1.1 Hiệu năng từng mô hình và tập hợp

Bảng 2 trình bày kết quả trên tập kiểm tra DongTing (lấy trực tiếp từ results.json). VAE đạt AUC kiểm định cao và ổn định qua ba hạt giống (0,9426 / 0,9438 / 0,9430; chọn hạt giống 1). Trên tập kiểm tra, tập hợp đạt AUC 0,835 và Average Precision 0,990.

5.1.2 Đối chiếu với số liệu công bố

Bảng 3 đối chiếu kết quả tái lập với mức công bố của DeSFAM [1]. Có *tái lập được* ở hai điểm: (i) AUC kiểm định của VAE thậm chí cao hơn mức công bố cho thành phần VAE; (ii) độ chính xác (precision) rất cao ($\approx 0,999$), khớp định hướng “ít báo động giả”. Tuy nhiên *chưa tái lập được* F1/Recall ở mức công bố: F1 tập hợp của chúng tôi 0,417 so với 0,92 trong bài báo, do recall thấp (0,264) tại ngưỡng bảo thủ chọn trên tập

Bảng 2: Hiệu năng SyscallAD trên tập kiểm tra DongTing (cửa sổ 15/3, 43 chiều). Ngưỡng chọn trên tập kiểm định.

Mô hình	AUC	AP	F1	Precision	Recall
iForest	0,851	0,991	0,474	0,999	0,311
VAE	0,734	0,985	0,421	0,999	0,267
Tập hợp	0,835	0,990	0,417	0,999	0,264
VAE (AUC kiểm định, hạt giống tốt nhất)	0,944	—	—	—	—

kiểm định (ngưỡng ưu tiên precision). Nói cách khác, mô hình *xếp hạng* bất thường tốt (AUC/AP cao) nhưng *điểm chốt nhị phân* tại một ngưỡng cố định lại bỏ sót nhiều cửa sổ tấn công.

Bảng 3: Tái lập so với số liệu công bố của DeSFAM (lỗi phát hiện bất thường).

Chỉ số	DeSFAM (công bố)	Tái lập	Nhận định
AUC (tập hợp)	0,94	0,835	Thấp hơn nhưng cùng bậc
AP	0,87	0,990	Cao hơn (dữ liệu mất cân bằng)
F1 (tấn công)	0,92	0,417	Chưa tái lập —do ngưỡng ưu tiên precision
VAE (AUC kiểm định)	0,747*	0,944	Cao hơn mức công bố cho VAE
Precision	cao	0,999	Tái lập định hướng ít báo động giả

*Giá trị thành phần VAE theo Bảng 1 của bài báo gốc; tập hợp công bố $\approx 0,94$.

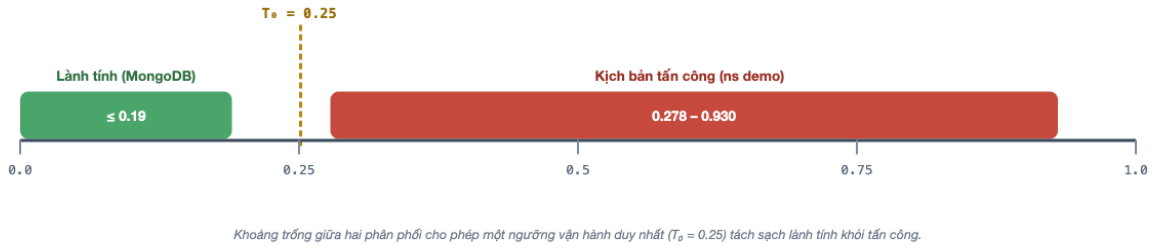
5.2 CH2 — Triển khai trực tiếp trên Tetragon/Kubernetes

Khi phục vụ trực tiếp, mô hình huấn luyện ngoại tuyến giữ được *khả năng phân tách* giữa lành tính và tấn công—kết quả quan trọng nhất của chuyên đề:

- Khối lượng lành tính (MongoDB) có điểm bất thường $\leq 0,19$.
- Các kịch bản tấn công trong không gian tên demo có điểm 0,278–0,930.

Khoảng trống giữa hai phân phối cho phép đặt một **ngưỡng vận hành** $T_0 = 0,25$ (lưu trong `inference/.env.uat`) tách sạch hai lớp trong môi trường thử nghiệm (Hình 3). Đây là bằng chứng cho thấy thiết kế DeSFAM khả thi ngoài mô phỏng: cùng một không gian đặc trưng 43 chiều, sinh ngoại tuyến từ DongTing và trực tuyến từ Tetragon, tạo ra điểm số tương thích.

Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon / Kubernetes)



Hình 3: Phân tách điểm bất thường khi phục vụ trực tiếp qua Tetragon trên Kubernetes. Khối lượng lành tính (MongoDB) nằm dưới ngưỡng vận hành $T_0 = 0,25$, các kịch bản tấn công nằm trên—khoảng trống đủ rộng để một ngưỡng duy nhất tách sạch hai lớp.

Bảng 4: Phân tách điểm bất thường khi phục vụ trực tiếp (Tetragon/K8s).

Nguồn	Khoảng điểm	So với $T_0 = 0,25$
MongoDB (lành tính)	$\leq 0,19$	Dưới ngưỡng (không báo động)
Kịch bản tấn công (demo)	0,278–0,930	Trên ngưỡng (báo động)

5.3 CH3 — Điều kiện tái lập thành công

Yếu tố quyết định là **căn chỉnh bảng chữ cái cảm biến** giữa huấn luyện và phục vụ. Khi huấn luyện trên toàn bộ vũ trụ syscall (gồm read/write/close/futex mật độ cao) trong khi Tetragon chỉ phát 23 syscall an ninh, mọi cửa sổ trực tiếp trở nên thừa và *suy biến*: điểm lành tính $\approx$ điểm tấn công, không thể đặt ngưỡng. Sau khi buộc `train.py` lọc DongTing về đúng 23 syscall trước khi cắt cửa sổ (`SENSOR_ALPHABET=1`), AUC kiểm định của VAE tăng lên 0,944 (giải quyết sụp đổ hậu nghiệm) và đạt được độ phân tách trực tiếp ở Bảng 4. Bài học: trong các hệ phát hiện bất thường *huấn luyện-rồi-phục-vụ*, sự đồng nhất không gian đặc trưng quan trọng ngang—thậm chí hơn—kiến trúc mô hình. Tính đồng nhất này được ràng buộc bằng kiểm thử ngang hàng `test_feature_parity.py` (đạt) để `train.py` và bộ trích đặc trưng khi phục vụ sinh véc-tơ *trùng khít từng bit*.

6 Thảo luận

6.1 Điều gì tái lập được, điều gì không

Chuyên đề tái lập được *định hướng và tính khả thi* của lõi DeSFAM: bộ phát hiện VAE + iForest xếp hạng bất thường tốt (AUC kiểm định VAE 0,944; AP tập hợp 0,990) và giữ khả năng phân tách khi phục vụ trực tiếp qua Tetragon. Điều *chưa* tái lập được là các con số tuyệt đối ở chế độ phân loại nhị phân—đặc biệt F1 0,92 và recall cao của bài báo. Nguyên nhân khả dĩ: (i) bài báo gốc chọn ngưỡng tối ưu F1 hoặc đánh giá trên một phân chia tấn công/khối lượng khác; (ii) khác biệt về phiên bản DongTing, cách cắt cửa sổ và tập syscall đầu vào; (iii) bài báo bổ sung Giai đoạn 3 (thực thi trong nhân) để đạt “chặn 14/14 CVE” —một chỉ số *thực thi*, không phải chỉ số phân loại của

riêng SyscallAD.

6.2 Ý nghĩa của khoảng cách ngưỡng

Việc AUC/AP cao nhưng F1 thấp tại ngưỡng cố định cho thấy giá trị của *hiệu chỉnh ngưỡng theo môi trường*. Ngưỡng bảo thủ chọn trên tập kiểm định (0,81 cho điểm tập hợp đã chuẩn hóa) tối đa hóa precision nên bỏ sót cửa sổ tấn công “nhẹ”. Ngược lại, khi phục vụ trực tiếp, ngưỡng vận hành $T_0 = 0,25$ được hiệu chỉnh theo phân phối quan sát giúp tách sạch hai lớp. Đây là một phát hiện thực dụng: với phát hiện bất thường, hiệu chỉnh ngưỡng tại chỗ quan trọng ngang với chất lượng mô hình.

6.3 Lựa chọn thiết kế khác bài báo

Chúng tôi dùng Tetragon [3] làm cảm biến eBPF thay vì tự viết chương trình eBPF/LSM. Điều này đẩy nhanh hiện thực hóa và bám hệ sinh thái Kubernetes thực tế, nhưng cũng *thu hẹp* bằng chữ cái syscall về 23 lời gọi an ninh—một ràng buộc kéo theo phải lọc dữ liệu huấn luyện tương ứng (Mục 5, CH3). Việc thực thi/chặn trong nhân (Giai đoạn 3 đầy đủ của DeSFAM) chưa được hiện thực hóa; chuyên đề dừng ở phát hiện và cảnh báo.

6.4 Đề dọa tính hợp lệ

- **Hợp lệ nội tại:** kịch bản tấn công trực tiếp là *mô phỏng* chuỗi syscall đặc quyền, không phải khai thác CVE thực; do đó độ phân tách ở Bảng 4 là dấu hiệu khả quan chứ chưa phải bằng chứng chặn khai thác thật.
- **Hợp lệ ngoại tại:** một khối lượng lành tính (MongoDB) và một không gian tên thử nghiệm chưa đại diện cho đa dạng khối lượng sản xuất; cần mở rộng (Nginx, Redis, Node.js...) như bài báo gốc.
- **Hợp lệ kết luận:** mất cân bằng lớp lớn (tỉ lệ tấn công cao trong tập kiểm tra cửa sổ) khiến AP cao dễ gây hiểu nhầm; chúng tôi báo cáo kèm AUC, precision, recall để tránh kết luận một chiều.
- **Hợp lệ cấu trúc:** đặc trưng thời gian bị tắt do DongTing không có dấu thời gian theo syscall; mô hình có thể thiếu một tín hiệu mà bài báo gốc tận dụng.

6.5 Hướng cải thiện

(1) Chọn ngưỡng tối ưu F1 và báo cáo đường cong precision–recall đầy đủ; (2) hiện thực hóa Giai đoạn 3 (chặn bằng eBPF/LSM hoặc Tetragon Enforcement) để đo “tỉ lệ chặn” so sánh trực tiếp với bài báo; (3) mở rộng tập khối lượng và dùng khai thác CVE thực trong môi trường cách ly; (4) bổ sung đặc trưng thời gian khi dùng cảm biến có dấu thời gian (Tetragon cung cấp), điều DongTing thiếu.

7 Kết luận

Chuyên đề đã tái lập độc lập phần lõi của DeSFAM [1]—bộ phát hiện bất thường lời gọi hệ thống SyscallAD (VAE + Rừng cô lập)—trên bộ dữ liệu công khai DongTing, và mở rộng nó thành một bộ phát hiện chạy trực tiếp bằng Tetragon/eBPF trên Kubernetes, toàn bộ đóng gói bằng Docker để bảo đảm tái lập.

Trả lời các câu hỏi nghiên cứu: **CH1**—tái lập được *định hướng và khả năng xếp hạng* (VAE AUC kiểm định 0,944, AP tập hợp 0,990, precision 0,999) nhưng *chưa* đạt F1/recall ở mức công bố do ngưỡng ưu tiên precision; **CH2**—mô hình giữ được khả năng phân tách khi phục vụ trực tiếp (lành tính $\leq 0,19$ so với tấn công 0,278–0,930), cho phép ngưỡng vận hành $T_0 = 0,25$; **CH3**—điều kiện then chốt là căn chỉnh không gian đặc trưng (bảng chữ cái cảm biến 23 syscall) giữa huấn luyện và phục vụ, được ràng buộc bằng kiểm thử ngang hàng.

Đóng góp chính của chuyên đề không phải một thuật toán mới mà là một *kiểm chứng tái lập trung thực*: chỉ rõ điều gì tái lập được, điều gì lệch và vì sao, đồng thời lấp khoảng cách “mô phỏng $\rightarrow$ triển khai” bằng một bộ phát hiện trực tiếp. Hướng tiếp theo gồm chọn ngưỡng tối ưu F1, hiện thực hóa khâu chặn trong nhân (Giai đoạn 3), mở rộng khối lượng công việc và dùng khai thác CVE thực trong môi trường cách ly.

Tài liệu tham khảo

- [1] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time,” *IEEE Access*, vol. 13, 2025.
- [2] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, “DongTing: A large-scale dataset for anomaly detection of the Linux kernel,” *Journal of Systems and Software*, vol. 203, p. 111768, Sep. 2023.
- [3] Cilium / Isovalent, “Tetragon: eBPF-based security observability and runtime enforcement,” <https://tetragon.io/>, 2024.
- [4] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, “A container security survey: Exploits, attacks, and defenses,” *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [5] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” in *IEEE Access*, vol. 7, 2019, pp. 52 976–52 996.
- [6] T. Combe, A. Martin, and R. Di Pietro, “To docker or not to docker: A security perspective,” *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [7] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on docker hub,” in *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
- [8] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, “Confine: Fine-grained system call filtering for container attack surface reduction,” *Computers & Security*, vol. 132, p. 103338, Sep. 2023.

- [9] S. Kim, B. J. Kim, and D. H. Lee, “Prof-gen: Practical study on system call whitelist generation for container attack surface reduction,” in *Proceedings of the IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sep. 2021, pp. 278–287.
- [10] S. Yang, B. B. Kang, and J. Nam, “Optimus: Association-based dynamic system call filtering for container attack surface reduction,” *Journal of Cloud Computing*, vol. 13, no. 1, p. 71, Mar. 2024.
- [11] J. Her, C. H. Jo, T. Park, and S. Lee, “KubeRosy: A dynamic system call filtering framework for containers,” *IEEE Access*, vol. 12, pp. 159 889–159 901, 2024.
- [12] S. Song, S. Suneja, M. V. Le, and B. Tak, “On the value of sequence-based system call filtering for container security,” in *Proceedings of the IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 296–307.
- [13] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A sense of self for Unix processes,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1996, pp. 120–128.
- [14] S. A. Hofmeyr, S. Forrest, and A. Somayaji, “Intrusion detection using sequences of system calls,” *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.
- [15] C. Warrender, S. Forrest, and B. Pearlmutter, “Detecting intrusions using system calls: Alternative data models,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1999, pp. 133–145.
- [16] D. Wagner and P. Soto, “Mimicry attacks on host-based intrusion detection systems,” in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002, pp. 255–264.
- [17] G. Creech and J. Hu, “A semantic approach to host-based intrusion detection systems using contiguous and discontiguous system call patterns,” *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
- [18] A. S. Abed, C. Clancy, and D. S. Levy, “Applying bag of system calls for anomalous behavior detection of applications in Linux containers,” in *Proceedings of the IEEE Globecom Workshops*, 2015.
- [19] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, “Deep autoencoding Gaussian mixture model for unsupervised anomaly detection,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [20] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 4393–4402.
- [21] W. Haider, J. Ahmad, M. Nasim, Z. A. Baig, and A. A. Laghari, “Methods for host-based intrusion detection with deep learning,” *Digital Threats: Research and Practice*, vol. 2, no. 4, pp. 1–29, 2021.

- [22] P. Castanhel, P. Pinto, and F. Magalhães, “Taking a peek: An evaluation of anomaly detection using system calls for containers,” in *Proceedings of the IEEE International Symposium on Computers and Communications (ISCC)*, 2021.
- [23] M. El Khairi, D. Colle, F. De Turck, and B. Volckaert, “Contextualizing system calls in containers for anomaly-based intrusion detection,” in *Proceedings of the ACM Cloud Computing Security Workshop (CCSW)*, 2022.
- [24] Q. Bsoul, M. Al-Quraan, and F. Al-Turjman, “Ensemble of random and isolation forests for graph-based intrusion detection in containers,” in *Proceedings of the IEEE International Conference on Cyber Security and Resilience (CSR)*, 2022.
- [25] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014.
- [26] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [27] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, “Mining sequential patterns by pattern-growth: The PrefixSpan approach,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 11, pp. 1424–1440, 2004.
- [28] B. Gregg, “BPF performance tools: Linux system and application observability,” *Addison-Wesley Professional*, 2019, iISBN: 978-0-13-655482-1.
- [29] Cilium Authors, “Cilium: eBPF-based networking, observability, and security,” <https://cilium.io/>, 2024.
- [30] NIST National Vulnerability Database, “CVE-2021-4034: Polkit local privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2021-4034>, 2022.
- [31] —, “CVE-2022-0847: Dirty pipe — linux kernel privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2022-0847>, 2022.
- [32] —, “CVE-2019-5736: Docker runc container escape,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.

A Phụ lục Tái lập Kết quả

Phụ lục này cho phép một người đánh giá độc lập tái lập kết quả. Mọi công cụ chạy bằng Docker—không cần cài đặt trực tiếp lên máy chủ (ngoài Docker và, cho phần trực tiếp, một cụm Kubernetes có quyền nạp eBPF).

A.1 Môi trường

```
# Yêu cầu: Docker; (phần trực tiếp) một cụm Kubernetes + Tetragon.
git clone <REPO_URL> && cd kma-chuyen-de
# Dữ liệu DongTing: nạp từ các zip trong DongTing/DB (xem README).
```

A.2 Huấn luyện ngoại tuyến (tái lập Bảng 2)

```
cd DeSFAM/training
docker compose -f docker-compose.pipeline.yml run --rm train
# Ket qua so + nguong duoc ghi vao ../model/results.json
```

A.3 Triển khai trực tiếp (tái lập Bảng 4)

```
cd DeSFAM/Kubernetes
bash tetragon/install.sh
kubectl apply -f tetragon/tracing-policy.yaml # 23 syscall an ninh
bash deploy.sh                               # workload lanh tinh + tan cong

cd ../inference
docker compose --env-file .env.uat up -d --build # detector + Prometheus + Grafana
# Grafana: http://localhost:3000 (admin/admin)
```

A.4 Kiểm tra tính nhất quán đặc trưng

```
cd DeSFAM
# Bao dam train.py va bo trich dac trung khi phuc vu sinh vec-to trung khi:
docker compose -f training/docker-compose.pipeline.yml run --rm \
  train python -m pytest tests/test_feature_parity.py -q
```

A.5 Biên dịch báo cáo này (Docker)

```
cd Report
./build.sh # chay XeLaTeX qua Docker texlive -> main.pdf
```